DoS Attacks & Network Defenses — Part 2

---

1. SSL/TLS Renegotiation DoS

The Asymmetry Problem

TLS was designed for security, not fairness. The handshake workload is severely unbalanced:

- **Server cost:** At least 10× more processing power than the client per handshake.
- **Server capacity:** 150–300 handshakes/second.
- **Client capacity:** Up to 1,000 handshakes/second per attacker.
- Conclusion: one attacker can overwhelm ~3–7 servers using only CPU-level effort.

How the Renegotiation Attack Works

TLS allows a client to request renegotiation at any time — this triggers a brand new handshake on an already-established connection:

- Attacker opens a valid TLS connection (bypasses basic connection filters).
- Attacker immediately sends a stream of renegotiation requests.
- Server must perform a full asymmetric-key handshake for each request.
- Server's CPU is exhausted; legitimate users cannot complete handshakes.

> **Why hard to detect:** Both the connection and each renegotiation request are fully legitimate protocol operations — they look identical to real traffic and bypass firewalls and rate filters.

SSL/TLS MITM via Renegotiation (Attack in Action)

Beyond DoS, the renegotiation flaw was also exploited for man-in-the-middle injection:

- Attacker sits between client and server, completing the initial handshake with the server on behalf of the client.
- When the real client connects, the attacker renegotiates and injects an altered request (e.g., a modified HTTP request) into the already-established server session.
- The server sees the injected content as part of the same trusted session — it cannot tell the renegotiation came from an intermediary.

Defense Mechanisms

- **Bad Prevention — Disable renegotiation:** Breaks legitimate use cases (e.g., mutual TLS client authentication mid-session).
- **Bad Prevention — Rate-limit TLS connections/handshakes:** Degrades service for real users; attacker simply slows down slightly.

- **Mitigation — SSL Accelerator:** Offload all cryptographic operations from the main CPU to a dedicated hardware card with its own cryptographic processors. Dramatically increases handshake throughput.
- **Mitigation — Key size awareness:** Moving from 1024-bit to 2048-bit RSA keys increases CPU usage 4–7×, making the asymmetry worse. Use hardware acceleration when deploying larger keys.

---

2. Slow DoS Attacks

Core Idea

Flood attacks are fast and noisy. Slow DoS takes the opposite approach:

- Goal: keep each per-request server resource (socket, thread, buffer) occupied for the maximum possible time.
- With enough slow connections open simultaneously, the server runs out of resources even at very low bandwidth.
- Extremely hard to distinguish from a slow legitimate user on a bad connection.

Slowloris — Slow HTTP Headers

Targets any server that waits for a complete HTTP request header before processing:

- Attacker opens many TCP connections and sends a partial but valid-looking HTTP GET header.
- The final CRLF (blank line) that signals end-of-headers is never sent.
- At regular intervals, the attacker sends one more header line (e.g., Accept-Language:...) to reset the server's idle timeout — keeping each socket alive indefinitely.
- Server holds the connection open waiting for the missing final header, tying up a worker thread/socket.
- Repeat across hundreds of connections — server's socket pool is exhausted.

> **Key insight:** Each header line sent prevents the server from timing out the connection. The attacker never needs to complete the request.

RUDY — Slow HTTP POST (R-U-Dead-Yet)

Targets web forms and any endpoint that accepts POST data:

- Attacker sends a legitimate HTTP POST with a very large Content-Length header (e.g., 10,000 bytes).
- The actual POST body is then sent one byte (or a few bytes) at a time, with long pauses between each chunk.
- The server must keep the connection open and buffer the incoming data, because based on Content-Length it knows more data is still coming.
- Server resources (connection, buffer, thread) remain tied up for as long as the attacker chooses.

> **Contrast with Slowloris:** Slowloris attacks the request header phase; RUDY attacks the request body phase. Both exploit the server's obligation to wait.

Slow Read Attack

Attacks from the opposite direction — the attacker is the reader, not the writer:

- Attacker issues a legitimate HTTP GET request for a large file (e.g., a large image).
- Attacker advertises an extremely small TCP receive window (e.g., 28 bytes) in its ACK packets.
- Because TCP respects the receiver window, the server can only send a tiny amount of data at a time and must wait for the window to open before sending more.
- The server holds the connection open, keeping the socket, send buffer, and associated state allocated for the entire slow transfer.
- Larger file = longer tie-up. Multiple such connections quickly exhaust the server.

---

3. Defense Against Slow DoS (Akamai Approach)

Slow GET / Slowloris Defense

- Deploy a proxy server at the network edge.
- The edge proxy waits for the complete HTTP header before forwarding anything to the origin server.
- The origin server only ever sees complete requests — it is shielded from partial/slow headers entirely.
- Slow connections are absorbed and terminated at the edge; the origin is never burdened.

Slow POST / Slow Read Defense

- The edge server monitors the bit rate of each connection over multiple measurement intervals.
- If the rate is consistently below a threshold (e.g., 10 bits/sec for more than 6 consecutive intervals = 30 seconds total), the connection is flagged and aborted.
- This catches both RUDY (slow incoming POST body) and Slow Read (slow ACKs from client).

> **Trade-off:** Thresholds must be tuned carefully — too aggressive kills legitimate users on slow connections; too lenient lets attacks through.

---

4. Hash DoS Attacks

The Hash Table Vulnerability

Web servers and frameworks commonly parse HTTP POST data (form fields) into a hash table for efficient lookup. Hash tables assume O(1) insertion on average — but this breaks under hash collisions:

- **Normal case:** Insert n elements → O(n) total time.
- **Collision case:** Insert into a chain → O(n) per insert → O(n²) total for n elements.

The Attack

An attacker crafts an HTTP POST request containing thousands of form field names that all hash to the same bucket:

- A malformed POST request under 2 MB can craft ~50,000 hash collisions.
- Processing this single request takes close to 30 minutes of CPU time (vs. milliseconds normally).
- One attacker, one machine, one request → server CPU pinned at 100%.
- **Affected platforms:** Java, Apache Tomcat, ASP.NET, PHP — any framework that uses unsalted hash maps for POST parsing.

> **Why it works:** The hash function used to map field names to buckets was deterministic and public. An attacker can precompute keys that all collide.

Defenses Against Hash DoS

- Limit number of POST variables per request (e.g., reject requests with > 1,000 fields).
- Enforce a CPU time limit per request — abort processing if it runs too long.
- Change the hash function seed frequently (randomise per-process or per-request) so the attacker cannot precompute collisions.
- Use cryptographically strong hash functions that are collision-resistant even with known inputs.

---

5. Network Defenses

IANA Port Numbering

Ports are divided into three ranges by the Internet Assigned Numbers Authority (IANA):

- **System / Well-Known Ports [1–1023]:** Reserved for common services. Require root/admin to bind. Examples: HTTP → 80, HTTPS → 443, SSH → 22, DNS → 53.
- **User / Registered Ports [1024–49151]:** Less well-known services registered by vendors.
- **Ephemeral / Dynamic / Private Ports [49152–65535]:** Short-lived ports assigned automatically by the OS to outbound connections (source ports).

Local Services — Exposure Risk

Not every service should be reachable from the public internet, even if it is running legitimately on a machine:

- **Recursive DNS Resolvers:** If exposed publicly, allow amplification DDoS attacks (attacker uses them as reflectors).
- **Windows File Sharing (SMB, TCP 445):** Historically riddled with remote code execution vulnerabilities; local machines may have weak or no passwords.

Best practice: expose only the minimum set of services required, and restrict everything else at the firewall.

---

6. Firewalls

What a Firewall Does

A firewall sits between a Local Area Network (LAN) and the Internet, inspecting packets and deciding which traffic is allowed to pass. It can be implemented as rules on a router or as a dedicated standalone device.

Basic Packet Filtering (Stateless)

Operates at the IP/Transport layer only. Decisions are made per-packet based on:

- IP source and destination address.
- Protocol (TCP, UDP, ICMP, etc.).
- TCP/UDP source and destination port numbers.

Example rule:

```
DROP ALL INBOUND PACKETS TO TCP PORT 445   (blocks Windows File Sharing
from internet)
```

The Stateless Filtering Problem

Stateless rules cause a fundamental problem for outbound connections:

- Internal clients make outbound connections (e.g., browse the web) from random high-numbered ephemeral source ports.
- Responses from the internet arrive inbound, destined for those ephemeral ports.
- A stateless rule 'DROP ALL INBOUND PACKETS IF DEST PORT != 80' blocks all those responses.
- Result: internal users cannot receive replies to their own requests.

Stateful Filtering

The solution: the firewall maintains a connection tracking table. When an internal host initiates an outbound connection, the firewall records the 5-tuple (src IP, dst IP, proto, src port, dst port) and automatically permits the corresponding inbound reply traffic.

- Only responses to established outbound connections are allowed in — unsolicited inbound traffic is still blocked.
- This is the standard behavior of all modern firewalls and home routers.

> **Example:** Internal client connects to telnet server on port 23 from ephemeral port 1234. Firewall notes this. Server reply to port 1234 is allowed through because it matches the tracked session.

Network Address Translation (NAT)

NAT maps an entire private internal network to a single public IP address. Most home routers perform NAT + stateful firewall simultaneously:

- Internal devices use private IP ranges (10.x.x.x, 172.16–31.x.x, 192.168.x.x) not routable on the public internet.
- When an internal device makes an outbound request, the NAT device rewrites the source IP to the public IP and records the mapping.
- Inbound responses are matched to the mapping table and forwarded to the correct internal host.
- Side effect: internal devices are invisible from the internet — they cannot receive unsolicited inbound connections (acts like a firewall by default).

Local (Host) Firewalls vs. Network Firewalls

- **Network firewalls:** Deployed at the edge of an organisation, protecting the entire LAN.
- **Host firewalls:** Run on individual machines (e.g., Linux iptables). Protect even against traffic that has already passed the network firewall (e.g., lateral movement within a network).
- Best practice: deploy both — defence in depth.

Example iptables rules (Linux):

```
sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j
ACCEPT
sudo iptables -A INPUT -p tcp --dport 22 -m conntrack --ctstate
NEW,ESTABLISHED -j ACCEPT
```

Application Layer Filtering

Deep packet inspection that understands specific application protocols — goes beyond port/IP filtering:

- SMTP (email): scan attachments for viruses; must decode MIME encoding, ZIP files, etc.
- HTTP: inspect POST bodies for SQL injection payloads, XSS, etc.
- Requires the firewall to reassemble and parse the full application-layer protocol, not just headers.

Outbound Traffic Inspection

Organisations also inspect traffic leaving the network, not just entering it:

- Block connections to known malicious IPs/domains.
- Prevent data exfiltration (e.g., sensitive files being uploaded).
- Block peer-to-peer protocols like BitTorrent.

> **Warning:** Some organisations install their own root CA certificates on employee machines to decrypt and inspect TLS traffic. This is technically possible but has significant privacy and trust implications.

---

7. Intrusion Detection Systems (IDS)

What an IDS Does

An IDS monitors network traffic (or host activity) for signs of attacks or policy violations. Unlike a firewall, an IDS does not block traffic — it detects and reports. Alerts are sent to a Security Information and Event Management (SIEM) system where analysts investigate.

Two Detection Methods

- **Signature Detection:** Maintains a large database of known attack patterns (rules). When traffic matches a rule, an alert is raised. Fast and low false-positive rate for known attacks, but blind to novel (zero-day) attacks.
- **Anomaly Detection:** Learns a baseline of normal behaviour (traffic volume, connection patterns, protocol usage) and alerts on deviations. Can catch novel attacks, but higher false-positive rate.

Major Open Source IDS Tools

- **Snort:** Rule-based, packet-level IDS. Widely deployed; large community rule sets. Example rule: alert icmp 192.168.1.10 any -> any any (msg: "ICMP Attempt Attack"; sid:1000005)
- **Zeek (formerly Bro):** Focuses on network traffic analysis and logging; programmable via its own scripting language. Better for building custom detectors.
- **Suricata:** Multi-threaded, high-performance; supports both signature and anomaly detection. Compatible with Snort rules.

Snort Rule Anatomy

A Snort rule has two parts — a Rule Header and a Rule Option:

```
alert  icmp  192.168.1.10  any  ->  any  any  (msg:"ICMP Attempt
Attack"; sid:1000005)
```

- **Action:** What to do — alert, log, drop, reject.

- **Protocol:** icmp, tcp, udp, ip.
- **Source Address / Source Port:** Who is sending.
- **Direction (->):** Direction of traffic to match.
- **Destination Address / Destination Port:** Who is receiving.
- **Rule Options:** Message to log (msg:), unique rule ID (sid:), content to match, etc.
